

AsymptoticAnalysis after eighteen reviews

Native delegation, evaluation contracts,
and reproducible incremental findings

Prepared for Vladimir Reshetnikov

Repository: `VladimirReshetnikov/Asymptotic`

Reviewed commit:

`6687962f3c858a4f93623cfc496f33e35c6763d4`

Abstract

The repository has developed beyond a specialized inverse-expansion package: it now combines real generalized asymptotic objects with a native Wolfram Language delegation layer. The strongest new findings in the inspected snapshot concern that boundary, not a newly disproved coefficient formula. Public native-kernel probes reproduce three distinct defects: configured backend defaults are ignored; a legal expansion variable named `Method` is misclassified when reconstructing native metadata; and a fully applied identity function changes automatic dispatch, causing otherwise supported requests to fail. Separate native evaluations of three narrow candidate patch groups repair the demonstrated behaviors while preserving selected controls.

A fourth contribution measures the storage amplification caused by eagerly retaining both a native series and its normalized expression. A fifth identifies a current README contradiction about automatic routing. The report compares the package with `Series`, `SeriesData`, `InverseSeries`, `ComposeSeries`, `Asymptotic`, and `AsymptoticSolve`, and independently verifies a representative irrational-exponent inverse in exact arithmetic. Previously catalogued defects and broad extension wish lists are not reissued as new findings. The accompanying archive contains reproducible Wolfram probes, fifteen desired-contract tests, a fail-closed candidate generator, exact Python oracles, and an evidence ledger. The combined patch and complete upstream test suite have not been validated here.

Native audit runtime: Wolfram Language 15.0.0, Linux x86-64,
kernel build dated 6 May 2026. Repository version declaration: 1.8.0.
Evidence is tied to the commit above, not to a moving `main` branch.

Contents

1	Executive assessment and evidence boundary	2
2	What the package adds to Wolfram Language	3
3	A controlled mathematical comparison	6
4	N01: configured backend defaults are ignored	7
5	N02: option-name variables lose their identity	9
6	N03: an applied identity function changes automatic dispatch	10
7	N04: eager native normalization has a measurable storage cost	12
8	N05: the README contradicts implemented automatic routing	13
9	A coherent repair design for the native boundary	13
10	Prioritized development opportunities	15
11	Artifacts, reproduction, and validation status	16
12	Conclusion	17
A	Source and novelty map	18
B	Reference conventions	18

1 Executive assessment and evidence boundary

1.1 What is new in this review

The package’s useful distinction is an explicit, manipulable model of selected real asymptotic germs, including ordered irrational powers, logarithmic blocks, selected exact inverse cores, retained source information, and separately scoped error evidence. That distinction should not be confused with a claim that Wolfram Language lacks asymptotic expansion or inversion. The new native layer already depends on, and deliberately exposes, substantial built-in functionality. Its **Native** result contract is appropriately different from the package’s analytic contract.[2, 3, 4]

The review register consolidates eighteen reports and 123 identified finding entries. It also records repairs made after the older snapshots, remaining proof obligations, and accepted compatibility work. A useful additional review must therefore identify a concrete delta, not count the same unimplemented idea again. The five entries below are the scope of the new finding ledger.[5, 6]

ID	Classification	Finding	Evidence
N01	Medium; API correctness	SetOptions changes the declared backend default, but dispatch ignores it. The alias has the same visible symptom. The difference changes the returned polynomial, not just an informational label.	Native baseline and candidate
N02	Medium; metadata and evaluation	A legal first positional rule using Method as the variable delegates correctly but loses variable metadata, so numerical application fails.	Native baseline and candidate
N03	Medium; automatic coverage	The presence of a fully applied Function anywhere in the held source overprotects the package route. An identity wrapper makes supported complex or native-option requests fail.	Native baseline and candidate
N04	Engineering opportunity	Mandatory eager Normal storage amplifies the native wrapper’s expression footprint. A measured case uses about 5.8 times the native payload’s ByteCount .	Native measurement; no speed or peak-memory claim
N05	Low; docu- mentation	The current README describes automatic routing near its top, but its development-status section says Automatic still uses the package engines.	Source contradiction and native routing control

N01–N03 are reproducible defects in the inspected implementation. They are not reports of mathematically false coefficients or false root certificates. N04 is a measured cost and a design opportunity, not proof of a leak. N05 is a specific current documentation error, not another general recommendation to write more documentation.

1.2 Snapshot, execution, and exclusions

The pinned commit is 6687962f3c858a4f93623cfc496f33e35c6763d4. Its commit timestamp is 10 September 2026 in UTC and 9 September in Los Angeles; the article date uses the latter. The package

is now loaded as `AsymptoticAnalysis.wl` into the `AsymptoticAnalysis'` context. Older reports and historical validation records may use earlier names. Their results are not silently relabelled as current acceptance.[1, 2, 4]

Source inspection covered the native bridge, the complete certificate arithmetic/controller, the exact special-function identity and fixed-parameter composition modules, the Dirichlet module, the standalone builder, and selected substantial sections of the core and series-operation modules. Documentation and the complete consolidated review crosswalk were examined. Focused original materials from reports 11 and 17 were checked to distinguish existing mathematical and performance discussions from the present deltas. The precise file-level scope is recorded in `evidence/source_coverage.json`. This is not a claim that every line in every kernel module, or every long prior article, was independently revalidated.

All public behavior labelled “native-confirmed” below was observed after loading the complete pinned standalone in a native Wolfram kernel. Candidate groups were tested separately against modified complete standalones. Several connector attempts returned network or upstream-service errors. These are not counted as package failures, successful tests, or evidence that a built-in cannot solve a problem. The preserved observations are transcriptions of successful connector evaluations, not fabricated command-line logs.

The existing correctness, performance, API, and extension registers remain important. Their already recorded findings are not reproduced here as new bugs. In particular, this article does not reissue the known precision-refinement, coefficient-ordering, nonuniform-composition, native analyticity-admission, certificate, sparse-export, or flat-sector issues. Nor does it re-propose the entire complex-sector, uniform-asymptotic, multivariate, or integration roadmap. The narrowly related register entries for each new contribution are identified where needed.[5, 7, 8]

2 What the package adds to Wolfram Language

2.1 Compare contracts and useful output, not function-name lists

`Series` already constructs Taylor, Laurent, fractional-power, and some logarithmic expansions; it supports expansion at infinity and successive variable specifications. Its usual output is `SeriesData`, and `Normal` removes that series structure. The `Analytic` option controls how unrecognized functions are treated. These are substantial existing capabilities, not gaps filled for the first time by this repository.[16]

`Asymptotic` infers asymptotic scales, admits finite or infinite real or complex centers, and supports term and approximation controls. Its documented input scope includes more than elementary expressions. `AsymptoticSolve` handles implicit equations and systems, with specified or unspecified dependent-variable centers and a real-domain form. An unsuccessful example would not establish a universal limitation of either solver.[18, 19]

The comparison becomes useful when it asks which representation, branch contract, precision model, and subsequent operations the caller needs. Table 2 makes those distinctions. The package columns summarize documented mechanisms; only the explicitly identified probes elsewhere in this article were independently executed in this review.[2, 9, 4]

Table 2: Capability and contract comparison. Native breadth and package analytic evidence are different axes.

Task	Built-in baseline	Repository contribution or boundary
Ordinary local expansion	Series and Asymptotic already produce extensive local expansions.	The package preserves a real coordinate, ordered blocks, remainder information, and provenance in one object. For routine Taylor output, direct native use may be simpler.
Irrational exponent support	A SeriesData exponent lattice is encoded by integer indices and a denominator. More general Wolfram expressions can also occur outside that simple encoding.	Sparse real-exponent blocks make an explicitly ordered incommensurate support available without pretending it is one ordinary rational lattice.
Series inversion	InverseSeries reverts a native series. AsymptoticSolve is a broader implicit-equation tool.	Selected real inverse engines expose generalized coefficients, complete block selection, branch information, retained inverse cores, and residual/refinement operations.
Series algebra	Native series participate in arithmetic, and ComposeSeries composes native series.	Package operations transport their own remainder contracts. Native wrappers are deliberately refused by analytic operations until an appropriate analytic contract is supplied.
Gamma and Barnes regimes	Built-in special functions and asymptotic machinery remain available and are used by parts of the package.	Specialized inverse scales retain exact Lambert-type cores and organize correction blocks in inverse core and inverse logarithmic quantities. This is a specific supported family, not unrestricted transseries inversion.
Flat and oscillatory behavior	Native results can contain exponentials, phases, and other non-polynomial expressions.	Selected package engines expose explicit sector/phase organization. Their finite admitted scope must not be mistaken for a general closure theorem.
Complex or successive-variable requests	Native Series/Asymptotic provide broader input and output conventions than the positive-real analytic package model.	Explicit native backends preserve complete native output in GeneralizedSeries , without promoting it into a package analytic remainder.
Numerical accuracy and certification	Native numerical approximation controls and equation-solving tools are useful but differ from the package's exact interval proof procedure.	InverseCertificate uses exact rational enclosures for an admitted explicit function and interval. A successful certificate has a narrower, stated meaning than global inverse correctness.

Task	Built-in baseline	Repository contribution or boundary
Other asymptotic tasks	Native documentation includes integration, transforms, differential-equation and related asymptotic workflows.	The public forward/inverse germ APIs are not replacements for the complete native asymptotics ecosystem. Existing adapter proposals belong to the prior roadmap, not this finding count.

The representation statement about `SeriesData` is precise: its exponent indices lie on a rational lattice when its coefficients are independent of the expansion variable. It does *not* say that all Wolfram outputs are restricted to such a lattice, nor that an expression containing irrational powers is necessarily unsupported by every built-in. Similarly, `InverseSeries` and `ComposeSeries` should be compared with operations on their intended native series inputs rather than with an arbitrary package object.[17, 20, 21]

2.2 The native wrapper is an intentional contract boundary

The bridge records the full `NativeResult`, the producing backend and runtime, the held original request, native order conventions, and a normalized expression. It stores missing analytic-remainder and exactness evidence rather than inferring those properties from a formal order term or the absence of one. This is a sound architectural distinction. It would be a regression to “fix” native operation refusals merely by manufacturing an infinite-precision analytic jet.[3, 4]

The ordinary package cutoff is exclusive: a request with cutoff h retains complete blocks of exponent strictly below h in its recorded local coordinate. Native `Series` includes the requested order in its usual Taylor convention. Specialized package scales can apply a cutoff inside an exact prefactor instead of to the complete expression. Therefore a feature or timing comparison must align the achieved approximation, not mechanically use the same integer everywhere. For example, the package’s ordinary cutoff 4 and native `Series` order 3 expose the same four Taylor coefficients of e^x ; package cutoff 3 intentionally does not include the cubic term.[10, 4, 16]

2.3 Evidence does not collapse into a single “success” flag

Four questions need separate answers: did an engine return an expression; does that expression carry a formal series order; is a real asymptotic estimate established on a stated approach; and is there a quantitative bound valid at a specified numerical target? The current native result contract explicitly avoids equating these levels. The certificate implementation also distinguishes the stored explicit forward function from an unspecified family represented by an input remainder.[4, 12]

This matters when assessing the new failures. N02 has a correct computed native expression and incorrect wrapper metadata. N03 is an avoidable refusal, not an incorrect polynomial. N01 selects an unintended valid contract and order convention. Tests that compare only `Normal` expressions, only failure tags, or only printed output would miss parts of these distinctions.

3 A controlled mathematical comparison

3.1 Forward expansion with incommensurate powers

Consider $x \rightarrow 0^+$ and

$$F(x) = \exp(x + x^{\sqrt{2}}).$$

The executed package request

```
AsymptoticExpansion[Exp[x + x^Sqrt[2]], {x, 0, 3},
  "Backend" -> "Package"]
```

returned the six blocks

$$1 + x + x^{\sqrt{2}} + \frac{x^2}{2} + x^{1+\sqrt{2}} + \frac{x^{2\sqrt{2}}}{2} + \mathcal{O}(x^3). \quad (1)$$

Here and below, short public symbol names in displayed examples assume the package has been loaded; the executable files use fully qualified names.

The formula has a direct independent proof. Put $t = x + x^{\sqrt{2}}$. Since $t = \mathcal{O}(x)$ on the positive approach, $e^t = 1 + t + t^2/2 + \mathcal{O}(x^3)$; expanding the finite square gives (1). Equivalently, its support is the set of pairs (j, k) with $j + k\sqrt{2} < 3$, and coefficient $1/(j!k!)$. The supplied Python oracle checks precisely that finite support in $\mathbb{Q}(\sqrt{2})$, with exact order comparisons.

On the audit kernel, the selected native call

```
Asymptotic[Exp[x + x^Sqrt[2]], {x, 0, 5}]
```

returned the original expression $e^{x+x^{\sqrt{2}}}$. That remains an exact expression for the function. It is not a wrong answer, and this observation does not prove that no other native formulation can expose the desired terms. The concrete usability distinction is that the package request exposed a complete sparse block set and a stated remainder frontier. The orders in the two calls are not asserted to be identical contracts.

3.2 Inverse coefficients and a stronger attained frontier

Now let $r > 1$ and solve $x + x^r = y$ on the positive branch near zero. The package probe used $r = \sqrt{2}$ and exclusive cutoff 3. An independent coefficient derivation is especially useful here because it avoids using the package's own residual implementation as the sole oracle.

Proposition 1. *Let $r > 1$ be fixed and $\delta = r - 1$. The positive inverse has, for each fixed $m \geq 0$,*

$$g(y) = y \sum_{k=0}^m c_k y^{k\delta} + \mathcal{O}(y^{1+(m+1)\delta}), \quad y \rightarrow 0^+, \quad (2)$$

where $c_0 = 1$ and, for $k \geq 1$,

$$c_k = \frac{(-1)^k}{k} \binom{rk}{k-1} = \frac{(-1)^k}{k!} \prod_{j=0}^{k-2} (rk - j). \quad (3)$$

The empty product at $k = 1$ equals 1.

Proof. Write $x = yU$ and $\varepsilon = y^\delta$. The defining equation becomes $U + \varepsilon U^r = 1$. In a neighborhood of $(U, \varepsilon) = (1, 0)$, the real power has an analytic local branch, and the derivative of the left side with respect

to U is 1 at that point. Thus U has an analytic expansion in ε . Set $v = U - 1$; then $v = -\varepsilon(1 + v)^r$. Coefficient extraction by Lagrange inversion gives

$$[\varepsilon^k]v = \frac{(-1)^k}{k} [z^{k-1}](1+z)^{rk} = \frac{(-1)^k}{k} \binom{rk}{k-1}.$$

The analytic Taylor remainder is $\mathcal{O}(\varepsilon^{m+1})$. Multiplication by y proves (2). Finally, $1 + rx^{r-1} > 0$ for $x > 0$, so the selected local positive solution agrees with the increasing positive inverse. $\square$

For $r = \sqrt{2}$ and cutoff 3, the executed package output was

$$\begin{aligned} g(y) = y - y^{\sqrt{2}} + \sqrt{2}y^{-1+2\sqrt{2}} + \left(-3 + \frac{1}{\sqrt{2}}\right)y^{-2+3\sqrt{2}} \\ + \left(-4 + \frac{17\sqrt{2}}{3}\right)y^{-3+4\sqrt{2}} + \mathcal{O}\left(y^{-4+5\sqrt{2}}\right). \end{aligned} \quad (4)$$

The attained frontier $-4 + 5\sqrt{2}$ is greater than 3; the preceding exponent $-3 + 4\sqrt{2}$ is less than 3. This is a useful example of a complete block boundary giving more information than an arbitrary requested threshold. The Python oracle checks the four displayed nonconstant coefficients and verifies $U + \varepsilon U^r - 1 = 0$ through order eight, independently, for $r = 3/2, 2, 3, \sqrt{2}$.

The native documentation supplies valid comparison forms such as

```
AsymptoticSolve[x + x^Sqrt[2] == y, x -> 0, {y, 0, 3}, Reals]
```

A preliminary exploratory call in this audit used a bare dependent variable, which is not used as evidence of native inability. Subsequent attempts to obtain a corrected comparison encountered connector errors and did not establish a capability result. Consequently, (4) is evidence that the package handled this input, not evidence that `AsymptoticSolve` cannot. The documented order parameter of `AsymptoticSolve` is not generally a polynomial degree or this package's exclusive exponent cutoff.[19]

4 N01: configured backend defaults are ignored

4.1 Reproduction and consequence

`SetOptions` changes the default options associated with a symbol. A wrapper exposing "`Backend`" in `Options` should either honor the configured default or clearly reject unsupported configuration; silently retaining an unrelated hard-coded default is misleading.[22]

In a fresh loaded kernel, execute:

```
a = AsymptoticExpansion[Exp[x], {x, 0, 3}];
SetOptions[AsymptoticExpansion, "Backend" -> "Series"];
b = AsymptoticExpansion[Exp[x], {x, 0, 3}];
c = AsymptoticExpansion[Exp[x], {x, 0, 3}, "Backend" -> "Series"];
{ {a["Kind"], Normal[a]}, {b["Kind"], Normal[b]},
  {c["Kind"], Normal[c]} }
```

The observed result was

```
{{"Forward", 1 + x + x^2/2},
 {"Forward", 1 + x + x^2/2},
 {"Native", 1 + x + x^2/2 + x^3/6}}
```


Changing the alias's default with `SetOptions[AsymptoticExpand, "Backend" -> "Series"]` also left its observed result on the `Forward` path.

The absent cubic term is not a bad Taylor coefficient or an off-by-one bug in either engine. It is the intended difference between package cutoff 3 and native order 3, exposed by the wrapper selecting the wrong backend relative to the user's configured default. That makes the defect more consequential than stale metadata: configuration changes which mathematical representation and order contract the user expects.

4.2 Source mechanism

The dispatcher collects explicitly supplied backend selectors and then uses

```
backend = If[values === {}, Automatic, ReleaseHold[First[values]]];
```

The empty explicit-selector case never consults `Options[AsymptoticExpansion]`. Separately, the alias has its own copied options list but delegates immediately to the main symbol. Its independently changed defaults are therefore not consulted at entry.[3]

This is a new concrete issue in the native dispatcher. The prior register already discusses cutoff conventions and other option-precedence problems, but neither a generic option-consistency warning nor a stored inverse observable's `Power` precedence describes this implementation path. The novelty is the observed default/explicit discrepancy and its direct order consequence.

4.3 Focused fix and tested behavior

The main-symbol candidate changes the empty-selector branch to

```
OptionValue[AsymptoticExpansion, {}, "Backend"]
```

The alias candidate appends a delayed default selector after the caller's arguments:

```
AsymptoticExpand[args___] := AsymptoticExpansion[args,
  "Backend" :> OptionValue[AsymptoticExpand, {}, "Backend"]];
```

The existing first-explicit-selector rule therefore wins; the appended default is consumed only when needed. This candidate deliberately chooses independently configurable `Backend` defaults for the two public heads. A genuinely shared-default alias is another coherent policy, but it should be specified rather than emerging accidentally from which entry point happens to resolve options.

A native evaluation of the text-modified complete standalone established the following observations in one request: a delayed main default selected `Native`; its normalized expression included $x^3/6$; an explicit `"Package"` request still selected `Forward`; the delayed default callback ran once in total across those two calls; and a configured alias default selected the `Asymptotic` backend. These are focused observations, not full option-system acceptance.

4.4 Why the broader fix is not just forwarding every default

The public options list is a union of package and native options. The native backends do not have identical option sets. For example, `Analytic` belongs to the observed `Series` options, whereas `Method` belongs to the observed `Asymptotic` options. Blindly forwarding every union default to every backend

would create new option conflicts. The current bridge also deliberately protects explicit package-only resource and branch restrictions.[3, 16, 18]

A durable design should resolve defaults in a backend-specific profile and retain their origin: explicit call, public entry-point profile, or selected native backend. Alias ownership must be settled for all options, not only **Backend**. Internal analytic replay should also be tested under a changed public backend default so that a repair does not unexpectedly route a precision-preserving internal construction into a native-only object. The shipped candidate is not represented as solving those wider questions.

The immediate workaround on the unmodified snapshot is to pass "**Backend**" explicitly on each affected call. The fifteen-test desired-contract file includes default, alias, explicit-priority, and delayed-callback cases to make the intended behavior executable.

5 N02: option-name variables lose their identity

5.1 A correct native result with unusable wrapper metadata

The symbol `Method` is a legal expansion variable in this executed native request:

```
Asymptotic[Sin[Method], Method -> 0]
(* Method *)
```

The explicit wrapper delegates the same computation correctly:

```
s = AsymptoticExpansion[Sin[Method], Method -> 0,
  "Backend" -> "Asymptotic"];
{s["NativeResult"], s["Variable"], s[1/10]}
```

But the observed wrapper result was

```
{Method, Missing["MultipleOrUnresolvedVariables"],
  Failure["NativeVariables", ...]}
```

The list-form control

```
AsymptoticExpansion[Sin[Method], {Method, 0, 1},
  "Backend" -> "Asymptotic"]
```

identified `Method` and evaluated at $1/10$ successfully. Thus the defect is not inability to use that symbol in native mathematics. It is loss of a positional role during wrapper metadata construction.

5.2 Context-free classification is the wrong abstraction

The predicate `nativeSpecificationQ` treats an arrow with symbolic left-hand side as a specification only when that symbol is not among the public option keys. But a rule's meaning depends on where it occurs. The first required argument after the source in this accepted native call is an expansion specification; a later `Method -> Automatic` can be an option. A global blacklist of option names cannot distinguish these roles.[3]

The wrapper uses the recovered specifications to determine its variable, then allows numerical application only for one identified symbol. That refusal is appropriate for genuinely multiple or unresolved variables. The error is that this request has exactly one identifiable variable, and the metadata extractor incorrectly

removes it.

An informative metamorphic property is alpha-renaming: replacing a fresh expansion symbol by another admissible fresh symbol should rename both the native result and the wrapper's variable metadata, provided no definitions or option positions change. The present implementation passes the ordinary-symbol instance and fails the `Method` instance.

5.3 Narrow patch and a proper parser

The candidate makes the first syntactically recognized positional specification authoritative when constructing the native result's metadata. It then retains the existing classifier for the remaining tail. The exact replacement is supplied in `patch_rules.json`. A native test of this candidate produced

```
{Method, Method, 1/10, x, True}
```

for, respectively, the native expression, recovered variable, numerical value, an ordinary `x` request with a real `Method` option, and a control asserting that multivariable numerical application still refuses.

This patch is intentionally a metadata repair, not a complete grammar rewrite. Other bridge functions still classify options and specifications during automatic routing. A production repair should parse the argument sequence once into positional specifications and option containers, then reuse that partition for routing, native invocation, and metadata. It must preserve multiple list specifications, delayed rules, nested option containers, and computed tails without evaluating the source a second time.

The tested workaround is to use the list specification for the colliding symbol, or use a different fresh variable. There is no reason to change the native expression or weaken the multivariable refusal to repair this case.

6 N03: an applied identity function changes automatic dispatch

6.1 Two equivalent sources, different accepted input classes

A fully applied pure identity is a particularly clean test because it introduces no inverse branch and no new mathematical operation. Nevertheless, these two calls behaved differently:

```
AsymptoticExpansion[Exp[I x], {x, 0, 3}]
AsymptoticExpansion[Function[z, z][Exp[I x]], {x, 0, 3}]
```

The first returned a `Native` object with

$$1 + ix - \frac{x^2}{2} - \frac{ix^3}{6}.$$

The second returned `Failure["InexactInput", ...]`, because it remained on the package's exact-real path.

A second witness does not require a complex source:

```
AsymptoticExpansion[Sin[x], {x, 0, 3}, Analytic -> True]
AsymptoticExpansion[Function[z, z][Sin[x]], {x, 0, 3},
  Analytic -> True]
```

The plain source returned `Native` with $x - x^3/6$. The wrapped source failed. Its failure tag was `UnsupportedOption`. Direct native `Series` of the identity-wrapped input returned the expected

SeriesData. Its failure message advised selecting a compatible backend or **Automatic**, even though the call was already using **Automatic**. This makes the overprotection visible both as lost coverage and as confusing recovery advice.

6.2 Source mechanism and novelty

The automatic protection predicate scans the entire held source for **_Function**, together with inverse functions, conditional sources, package series, and remainder descriptors. Such a scan treats a fully applied identity's function head as if the input were an unapplied callable whose branch/evaluation treatment must stay with the package. It is the *presence* of the syntactic node, rather than the semantic role of the source, that changes the route.[3]

The prior B01/B02 compatibility work already acknowledges incomplete automatic coverage. This finding is a sharper delta, not a repetition of that general admission: it identifies an exact overbroad predicate, gives evaluation-equivalent public witnesses, demonstrates a native-only-option failure on an elementary real source, and tests a bounded repair. It does not ask for a second-backend search or for abandonment of branch protection.

6.3 Candidate: protect unapplied callables, not every occurrence

The candidate removes **_Function** from the whole-source scans and adds explicit tests for an unapplied function at the top level of the held source. It retains whole-source protection for inverse-function syntax, conditional expressions, existing package series/remainers, the package's retained callable marker, and the existing explicit restriction checks.

Separate native tests of the modified complete standalone established all of the following: the identity-wrapped complex source routed to **Native** with the correct polynomial; the real identity source with **Analytic -> True** routed to **Native**; the unapplied **Function[z, Sin[z]]** remained **Forward**; and an explicit **Direction -> "FromAbove"** still protected the complex request and produced a refusal. A stateful applied identity incremented its counter exactly once while the fallback returned **Native**.

The counter control matters. A tempting implementation would evaluate the source to discover its role and then evaluate the original source again when invoking the native engine. The present bridge already has preparation/replay machinery intended to avoid repeating the original source program on fallback. A repair should reuse that machinery, not add an independent speculative evaluation.[4, 3]

The narrow candidate is not a theorem that all evaluation-equivalent Wolfram programs should have identical observable behavior. Held expressions, upvalues, side effects, and user definitions can make that an unreasonable demand. The tested property is deliberately smaller: a completely applied immediate identity, with no additional protected mathematical contract, should not by itself block a native-compatible request.

6.4 Development consequence

Route restrictions should be attached to the obligations they preserve. An unapplied inverse-function request has branch obligations. An explicit resource limit has an enforcement obligation. A native formal result has a representation contract. A spent identity-function application has none of those obligations merely because its original syntax contained **Function**. Recording these reasons explicitly would make both fallback policy and diagnostics less brittle.

On the baseline, selecting **"Backend" -> "Series"** explicitly bypasses the defective automatic classification when native behavior is intended. Removing a harmless wrapper also avoids this particular

witness, but users should not have to reverse-engineer a whole-source syntax blacklist.

7 N04: eager native normalization has a measurable storage cost

7.1 Measurement and its limits

The native constructor evaluates the selected backend, immediately computes `Normal[result]`, and stores both the complete native object and the normalized expression. This happens even when the caller's next operation only asks for `s["NativeResult"].[3]`

The measured family was

```
Series[1/(1 - x - y), {x, 0, n}, {y, 0, n}]
```

compared with the explicit "Series" wrapper using the same specifications. In each measured case the stored native result was exactly equal to the direct result, and the normalized forms were exactly equal. The structural storage measurements were:

n	Native bytes	Normal bytes	Wrapper bytes	Wrapper/native
8	3,936	11,312	20,824	5.29
16	10,528	42,160	60,248	5.72
24	20,192	92,464	117,832	5.84

Table 3: Native Wolfram 15.0.0 `ByteCount` observations. These are not process-memory or peak-allocation measurements.

`ByteCount` describes expression storage, and sharing and representation details affect its interpretation. The table is not a measurement of uniquely allocated resident memory, garbage-collector retention, or a leak. It nevertheless establishes that this API representation has a materially larger reported footprint than its native payload in the tested family.[23]

Timings are retained in `evidence/native_storage.json`, but they are *not* used to claim that one route is faster. Each direct `Series` call preceded `Normal` and the wrapper in the same kernel; the native cache was not cleared between those paired calls. Several wrapper times were therefore smaller than the preceding direct-call times. Treating those numbers as a clean wrapper-versus-native benchmark would be misleading.

7.2 Why this is an additional finding rather than the old performance list

P07/P08 already recommend profiling retained state and normalization. The new contribution is an actual measurement at the newly implemented native boundary, with an identified mandatory materialization step. It does not reissue the earlier unmeasured claim about potentially exponential recipe histories, nor the already addressed sparse-to-dense analytic exporter.[5]

For this example, the successive truncations expose $(n + 1)^2$ coefficient positions. A dense native representation and an ordinary expression have different node overhead even when they describe the same coefficients. The magnitude and bit length of the binomial coefficients also grow with order, so a simple term-count argument is not a complete asymptotic byte-complexity analysis. The evidence supports prioritizing measurement of duplicated representation, not a universal constant-factor claim.

7.3 An improvement with an explicit semantic choice

A native-payload-first construction mode could avoid materializing the normalized expression until the caller requests it. This is especially useful for callers that continue native algebra on **NativeResult**, inspect specifications, or store an intermediate result without needing ordinary expression form.

The design must state when normalization happens and under which evaluation context. Replacing an eager value by a delayed rule is not automatically semantics-preserving in Wolfram Language: later definitions, option values, or side effects can affect evaluation. A safe first implementation is therefore an *explicit opt-in* materialization policy, rather than a silent change to the existing **Expression** property. It should document whether **Normal[s]** computes on demand, whether that result is cached, and which provenance is retained.

A focused acceptance experiment should compare equal native payloads at matched specifications, measure construction and first/repeated normalization separately, randomize or isolate cache state, and record both structural **ByteCount** and process-level peak memory when an appropriate local runtime is available. No performance patch or speedup is claimed in this archive. The supplied measurement harness is the starting artifact.

8 N05: the README contradicts implemented automatic routing

The current README's development-status section says that both public expansion names currently use the package engines under "**Backend**" -> **Automatic**. The same snapshot documents automatic native routing elsewhere, and the executed plain **Exp[I x]** request returns **Native** with no explicit backend. The development-status sentence is therefore not an accurate description of this snapshot.[2, 4]

The nearby count of 130 successful focused checks belongs to an explicit-native acceptance record; its historical existence is not disputed here. The error is to let a stale status paragraph imply that automatic dispatch is still wholly unimplemented, or to treat any historical focused count as acceptance of every later change. The contract document itself is careful about this distinction.[4]

A replacement paragraph should say that **Automatic** preserves successful package requests, routes native specifications/options where allowed, and falls back after selected representation failures while protecting explicit restrictions. It should link to the current routing contract and its scoped evidence, while leaving the still-unproved full input-superset objective clearly open.

For this specific page, two short executable documentation controls are enough to prevent the observed contradiction: an ordinary real Taylor request should document its package result, and a plain complex request should document its automatic native result. Store the producing commit/runtime beside any checked status snippet. This is a concrete documentation repair for the bridge, not a request to repeat the entire prior CI roadmap.

9 A coherent repair design for the native boundary

9.1 One parsed request, several distinct decisions

N01–N03 share a design pressure: the bridge currently answers configuration, syntax, mathematical-obligation, and metadata questions with partly separate scans. Repairing each symptom is useful, but a single parsed request would reduce the chance of contradictory answers.

The following is a proposed internal record, not an API already implemented in the repository:

```
<| "PublicHead" -> AsymptoticExpansion,
  "OriginalArguments" -> HoldComplete[...],
  "SourceSyntax" -> HoldComplete[...],
  "ArgumentRoles" -> {...},
  "ExplicitOptions" -> {...},
  "BackendDefaultOwner" -> AsymptoticExpansion,
  "SelectedBackend" -> ...,
  "SelectionOrigin" -> "Explicit" | "Default" | "Automatic",
  "PreservedObligations" -> {...},
  "PreparedSource" -> ... | Missing["NotEvaluated"] |>
```

The source need not be evaluated merely to create this record. Computed trailing containers may require a later preparation stage, and their eventual role should be recorded instead of retroactively guessed from an option-name blacklist. A dynamic request can remain unresolved until the existing evaluation boundary supplies the required value.

This proposal is more specific than the existing general result-contract/capability discussion in D02. It is designed around the three reproduced errors: default ownership is explicit for N01; a rule’s positional role is reused for N02; and a restriction’s reason is separated from an incidental syntax occurrence for N03.

9.2 Resolve backend profiles without losing restrictions

Backend selection should first distinguish an explicit selector from a configured default. After selection, resolve only the options relevant to that backend, using a declared policy for inherited native defaults. Keep the package’s own options separate from native engine options whose names happen to be advertised through the same public wrapper.

Explicit restrictions must not vanish during fallback. For example, if the native route cannot honor a package resource restriction, it should refuse that route and say which restriction is unsupported. Conversely, the mere presence of a consumed identity function should not manufacture a real-analytic restriction that the caller did not request. Both properties can coexist; N03’s candidate controls demonstrate this for a small, useful subset.

An automatic selection result should state whether it came from a native specification shape, an option that selects a native route, or a package representation failure. An “unsupported option” diagnostic should not recommend the very automatic setting that already produced the failure without explaining the protection reason. The current metadata already contains useful routing fields; the proposal is to make all stages consume the same reasoned selection rather than adding another independent reporting layer.[3, 4]

9.3 Metamorphic contracts that follow directly from the findings

The new regression corpus should include relations, not only individual expected polynomials:

1. **Default parity.** Configuring backend b and omitting the selector should agree in selected engine and mathematical result with explicitly selecting b , except for intentional provenance differences. Delayed defaults should be consumed once, and not consumed when overridden.
2. **Role-preserving renaming.** Renaming an admissible fresh expansion variable should rename the native result and recovered variable without changing which trailing rules are options.
3. **Spent-identity invariance.** A completely applied immediate identity should not add a branch restriction. A stateful preparation control should show that the repair does not replay the source

program.

4. **Restriction preservation.** The same tests with an explicit protected direction or resource restriction must not silently acquire unrestricted native behavior.
5. **Payload fidelity.** The full native result, rather than only its `Normal` form, should match the direct selected backend where delegation is intended.

These relations target newly demonstrated failures. They do not establish the complete native input-superset objective, and they do not imply formal equivalence of arbitrary Wolfram programs.

9.4 Keep expression access and analytic evidence separate

The native bridge already refuses to infer an analytic remainder from `SeriesData`. That decision should survive all repairs above. A correct parser is not a proof of uniformity, a successful native call is not an interval certificate, and numerical substitution into a native wrapper does not acquire the package's positive-real domain checks.

Likewise, a future lazy-normalization option should be an expression-materialization policy, not a request to weaken or strengthen a remainder. Provenance, selected order convention, source assumptions, and evidence type should remain available even when the ordinary expression is not yet materialized. This separation lets the implementation reduce overhead without accidentally changing what has been established mathematically.

10 Prioritized development opportunities

The following ordering follows the new evidence rather than reproducing the prior register's full backlog.

First: decide and repair default ownership. Adopt either independent public-entry defaults or genuinely shared alias defaults, then implement that policy consistently. The supplied backend patch is a small candidate for the former. Test explicit precedence and delayed evaluation before extending the change to other options. Include an analytic replay control under a changed default: the public policy must not cause an internal routine to return a native-only result where a precision-tracked analytic object is required.

Second: unify positional parsing and protection reasons. Promote the N02 metadata patch into a request parser reused by the routing and reporting paths. Combine it with N03's distinction between unapplied callables and consumed function applications. The immediate done criteria are the documented failures repaired, the selected controls preserved, and no additional source evaluation. A new global syntax scan should not be introduced merely to compensate for another scan's limitations.

Third: publish the bridge's executable behavior catalog. For each small input family, retain the direct native request, wrapper request, producing runtime, native payload equality, selection reason, and recovered specifications. Classify a timeout or unresolved call separately from a computed result. The catalog can include alpha-renamings, equivalent list/arrow forms, and explicit/default backend variants generated from a small grammar. A counterexample reducer should minimize only within that controlled grammar; it need not execute arbitrary untrusted user programs.

This is an immediately useful development artifact even before a complete compatibility theorem exists. It also makes documentation status precise: examples can show both successful package retention and successful automatic native routing without claiming every native form is covered.

Fourth: investigate native materialization as its own cost center. Use the N04 harness to measure native payload construction, wrapper metadata, eager ordinary-expression construction, first access, and repeated access separately. Only then select an opt-in delayed mode or another representation change. Preserve raw observations and cache state. Do not infer a speedup from the cache-confounded times in the present evidence.

Fifth: retain small independent coefficient oracles. The $x + x^r$ family provides an exact, analytically justified inverse oracle with a genuinely irrational support when $r = \sqrt{2}$. It tests coefficient algebra, complete block selection, and an attained frontier without relying on the package’s own residual code. The provided standard-library implementation over $\mathbb{Q}(\sqrt{2})$ is small enough to inspect. It complements native comparisons: agreement with a built-in is useful, but agreement with an independently derived identity answers a different question.

Broader custom scale extensions remain subject to the existing X-register’s mathematical obligations. Nothing in the new evidence justifies reprioritizing a large new special-function catalog ahead of these concrete bridge repairs. Equally, fixing the bridge does not close the already recorded mathematical correctness work.[5]

11 Artifacts, reproduction, and validation status

11.1 What is in the archive

The article is self-contained LaTeX with an accompanying compiled PDF. The `code` directory contains a pinned-source loader, an observation harness, fifteen desired-contract Wolfram tests and a focused runner, exact Python coefficient/support oracles, a candidate generator, and the five textual edits as data. The `evidence` directory records baseline observations, separate candidate observations, storage measurements, novelty mapping, source coverage, and executed Python logs.

The candidate generator requires a Git checkout whose `HEAD` matches the reviewed commit and whose canonical native module is clean. It checks all expected source anchors before replacing any of them, writes to a new output directory, and does not edit the checkout. The candidate includes a patched module and a unified diff. Apply it only in a reviewed worktree, regenerate the standalone using the repository’s own builder, and then run the focused tests on the generated candidate. The standalone builder already exists; this article is not proposing it as a missing feature.[15]

```
python code/independent_oracles.py
python -m unittest discover -s code -p test_patch_mechanics.py -v
python code/make_candidate.py /path/to/pinned/Asymptotic --out candidate

# In a fresh native kernel, using the pinned download by default:
wolframscript -file code/characterize.wl
wolframscript -file code/run_regressions.wl
```

For a reviewed local standalone, set `ASYMPTOTIC_AUDIT_SOURCE` to its absolute path. The loader records that override and does not claim to verify its version. Loading either local or downloaded Wolfram code executes it; review that source first.

11.2 What ran and what did not

Activity	Status
Pinned full-package probes	Successful native observations for N01–N03, the irrational forward/inverse example, and the N04 storage family.
Candidate N01 group	Tested separately in a text-modified complete standalone; configured main/alias backend and delayed/explicit-priority observations recorded.
Candidate N02 group	Tested separately in a text-modified complete standalone; colliding-variable repair, actual-option and multivariable controls recorded.
Candidate N03 group	Tested separately in a text-modified complete standalone; repaired identity requests, callable/direction controls, and one-evaluation counter recorded.
Combined candidate	Not run end-to-end. Separate group results do not establish their combined integration.
Packaged Wolfram harness and fifteen-test file	Supplied for reproduction; the complete command-line wrappers and the complete desired-contract suite were not executed as such. Equivalent focused expressions were evaluated through the native connector.
Independent Python mathematical tests	Six test methods passed, including four order-eight implicit-substitution cases within one parametrized test. Exact arithmetic; no native package dependency.
Python patch-mechanics tests	Five tests passed. These use explicit anchor fixtures, not a locally mounted complete upstream module.
Full upstream test suite	Not run. No aggregate release acceptance is claimed.
Performance	Structural storage observations only; no peak-memory result, leak finding, matched speed comparison, or implemented optimization.

The distinction between group-level native evidence and complete-suite acceptance is important. The candidate generator makes the edits reproducible; it does not certify their correctness. A passing fixed witness would also be insufficient to establish every advertised native argument form. Conversely, connector errors in unrelated attempts are not valid reasons to count an input as unsupported by Wolfram Language.

12 Conclusion

The inspected snapshot has a substantive mathematical identity: selected real generalized expansions and inverses with explicit block, branch, and evidence structure. It also has a newer interoperability identity: a wrapper around native expansion engines. The latter now deserves the same precision of contract as the former.

The three reproduced runtime defects are small enough to explain exactly and consequential enough to repair: backend defaults must influence selection; positional variable roles must survive metadata reconstruction; and a consumed identity function must not manufacture a branch restriction. The separately tested patch groups show practical ways forward without replacing native payloads or weakening analytic contracts. The storage measurements identify a further concrete design decision, and the README contradiction can be corrected immediately.

The recommended next increment is not another sweeping rewrite or another unqualified list of possible extensions. It is a coherent native request model, scoped integration of the reproduced fixes, and executable evidence that records both useful mathematics and the exact contract under which it was obtained.

A Source and novelty map

Area	Principal inspected source and role
Dispatch and native metadata	<code>src/Kernel/NativeCompatibility.wl</code> : complete bridge; default selection, protection predicate, specification recovery, native result construction.
Core mathematics and entry	<code>src/Kernel/AsymptoticAnalysis.wl</code> : selected public/assumption/comparison, forward/native-import, coordinate, formatting, residual, coefficient, and loader sections.
Derived series operations	<code>src/Kernel/SeriesOperations.wl</code> : representation, binary algebra, observable/composition, differentiation, and refinement sections.
Certificates	<code>src/Kernel/InverseCertificates.wl</code> : exact arithmetic, domain/branch checks, residual enclosure, and adaptive controller.
Special-function admission	<code>SpecialFunctionIdentities.wl</code> , <code>ParameterizedSpecialFunctions.wl</code> , and <code>DirichletSpecialFunctions.wl</code> : exact identities, finite-argument composition, and defining-sum bounds.
Distribution	<code>validation/build_standalone.py</code> : canonical-module assembly, validation, and freshness checking.
Prior-review exclusion	Complete consolidated status/crosswalk, report indexes, and focused original R11/R17 evidence. No claim of rerunning the eighteen historical audits.

N01 is related to prior general option/API discussions but adds the new dispatch-default witness. N02 is a concrete new native metadata failure. N03 sharpens the known B01/B02 compatibility gap with a precise lexical cause and tested controls. N04 supplies measurements missing from the related P07/P08 recommendations. N05 concerns the current bridge’s status text. This mapping is also machine-readable in `evidence/novelty_ledger.json`.

B Reference conventions

Repository references below point to the reviewed commit. Function names in the discussion identify the relevant source locations even if later versions move line numbers. Official Wolfram documentation was consulted during this review; native observations identify the actually used kernel rather than assuming that every installed version behaves identically. The mathematical derivation in Section 3 and the independent tests are part of this review, not claims copied from a prior report.

References

- [1] Asymptotic repository, reviewed commit record.

<https://github.com/VladimirReshetnikov/Asymptotic/commit/6687962f3c858a4f93623cfc496f33ec35c6763d4>

- [2] Asymptotic repository, README, including version declaration and development status.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/README.md>

- [3] Asymptotic repository, native compatibility implementation.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/src/Kernel/NativeCompatibility.wl>

- [4] Asymptotic repository, native expansion result contracts.

https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/docs/development/NATIVE_RESULT_CONTRACTS.md

- [5] Asymptotic repository, consolidated code-review implementation register and complete finding crosswalk.

https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/docs/development/CODE_REVIEW_STATUS.md

- [6] Asymptotic repository, external code-review index.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/external-reports/code-review/README.md>

- [7] External report 11, finding-delta ledger and reproduction harness.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/external-reports/code-review/wave-2/code-review-11/evidence/findings-delta.json>

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/external-reports/code-review/wave-2/code-review-11/code/reproduce.wl>

- [8] External report 17, novelty matrix.

https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/external-reports/code-review/wave-2/code-review-17/evidence/novelty_matrix.json

- [9] Asymptotic repository, kernel source map.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/src/Kernel/README.md>

- [10] Asymptotic repository, canonical package entry point and core.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/src/Kernel/AsymptoticAnalysis.wl>

- [11] Asymptotic repository, explicit series operations.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/src/Kernel/SeriesOperations.wl>

- [12] Asymptotic repository, inverse certificates.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/src/Kernel/InverseCertificates.wl>

- [13] Asymptotic repository, exact special-function identities and fixed-parameter composition.

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/src/Kernel/SpecialFunctionIdentities.wl>

<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33ec35c6763d4/src/Kernel/ParameterizedSpecialFunctions.wl>

- [14] Asymptotic repository, Dirichlet special-function expansions.
<https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e5c6763d4/src/Kernel/DirichletSpecialFunctions.wl>
- [15] Asymptotic repository, standalone builder.
https://github.com/VladimirReshetnikov/Asymptotic/blob/6687962f3c858a4f93623cfc496f33e5c6763d4/validation/build_standalone.py
- [16] Wolfram Research, *Series*. Official Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/Series.html>
- [17] Wolfram Research, *SeriesData*.
<https://reference.wolfram.com/language/ref/SeriesData.html>
- [18] Wolfram Research, *Asymptotic*.
<https://reference.wolfram.com/language/ref/Asymptotic.html>
- [19] Wolfram Research, *AsymptoticSolve*.
<https://reference.wolfram.com/language/ref/AsymptoticSolve.html>
- [20] Wolfram Research, *InverseSeries*.
<https://reference.wolfram.com/language/ref/InverseSeries.html>
- [21] Wolfram Research, *ComposeSeries*.
<https://reference.wolfram.com/language/ref/ComposeSeries.html>
- [22] Wolfram Research, *SetOptions*.
<https://reference.wolfram.com/language/ref/SetOptions.html>
- [23] Wolfram Research, *ByteCount*.
<https://reference.wolfram.com/language/ref/ByteCount.html>